



Lebanese University
Faculty of Engineering III
Electrical and Electronic Department

6Π ROBOTIC ARM

COMMUNICATION AND MINI-PROJECT

by

Hadi Kansa 6883

Ali Haydoura 6951

Spring 2025-2026

Supervisor:

Dr. Rida Chall

Dr Abed Latif Samhat

ACKNOWLEDGMENT

All praise and gratitude are due to Allah for granting us the strength, patience, and determination to complete this project successfully.

We would like to express our sincere gratitude to each member of our team , for their dedication, collaboration, and continuous support throughout this project. This work is the result of our collective effort, and every member contributed equally to its success.

We extend our deepest appreciation to our supervisor, Dr. Rida Chall.

Special thanks go to the Faculty of Engineering III at the Lebanese University, particularly the Electrical and Electronic Department

We extend our sincere gratitude to our families for their unwavering encouragement and financial support, which made it possible to acquire the required components such as the MG996R servos, ESP32 board, and PCA9685 driver. Their patience during the long hours of coding, testing, and debugging has been our motivation throughout this journey.

Finally, we acknowledge the open-source community, particularly the developers of the Adafruit PWMServoDriver, WebSocketsClient, ESP32(Espressif) and ArduinoJson libraries, whose contributions simplified the IoT implementation significantly.

Thank you all.

ABSTRACT

This paper presents the design and implementation of a **5-degree-of-freedom IoT**-controlled **robotic arm** weighing approximately **900 grams** with a **34.7 cm** reach. The mechanical structure was designed in SolidWorks and **3D printed** using PLA filament. Actuation uses six **servo motors** (four MG996R, two SG90) controlled by an **ESP32-S3** microcontroller via a PCA9685 PWM driver.

The system operates entirely over the Internet using **WebSocket** protocol, with the **ESP32** connecting to a **Microsoft Azure cloud server**. A web interface allows users to control individual **servos** via sliders or execute coordinated multi-servo batch commands. A non-blocking state machine executes grouped movements in three sequential stages: Group 1 (motors 1-2), Group 2 (motors 3-5), and Group 3 (motor 6), with 500-millisecond delays between groups. (This sequencing reduces peak current)

Movement is smoothed using 1-degree steps every 15 milliseconds.

Experimental results demonstrate reliable **WebSocket** communication with sub-100-millisecond latency, accurate servo positioning across 0-180 degrees, and smooth coordinated motion. The platform is architecturally prepared for future **inverse kinematics** integration. This project provides an affordable, open-source, remotely accessible robotic arm suitable for educational robotics laboratories.

Keywords: Robotic arm, 5-DOF, IoT, ESP32, WebSocket, Servo control, Inverse kinematics, 3D printing

TABLE OF CONTENTS

ACKNOWLEDGMENT	II
ABSTRACT	III
LIST OF FIGURES	3
LIST OF TABLES	4
CHAPTER 1 INTRODUCTION	5
1.1 Background Information	5
1.2 Problem Statement	6
1.3 Objective and Motivation	7
1.4 Thesis Outline	7
CHAPTER 2 LITERATURE REVIEW	8
2.1 Introduction	8
2.2 Robotic Arm Kinematics	8
2.3 Servo Motor Control	8
2.4 ESP32 Microcontroller	8
2.5 IoT Communication Protocols	9
CHAPTER 3 SYSTEM DESIGN	10
3.1 Introduction	10
3.2 System Requirements	10
3.2.1 Functional Requirements	10
3.2.2 User Interactions and System Responses	10
3.2.3 Mechanical Requirements	12
3.2.4 Software Requirements	13
3.2.5 Use Case Diagram	13
3.3 System Architecture	14

3.3.1 WebApp Architecture	14
3.3.2 Mechanical Architecture	16
3.4 Non-Functional Requirements	16
3.5 Conclusion	17
CHAPTER 4 IMPLEMENTATION AND RESULTS	18
4.1 Introduction	18
4.2 Implementation Tools	18
4.2.1 Software Tools	18
4.2.2 Hardware Tools	18
4.3 Implementation Steps	19
4.3.1 Mechanical Design	19
4.3.2 Circuit Assembly	22
4.3.3 Software Development	25
4.4 Results	26
4.4.1 Mechanical Results	26
4.4.2 Software Results	27
4.4.3 Overall Assembly and Final Results	28
CHAPTER 5 CONCLUSION	30
5.1 Conclusion	30
5.2 Future work	31
REFERENCES	32
APPENDICES	33

LIST OF FIGURES

Figure 3.1 : Complete CAD model of the robotic arm	12
Figure 3-2 : Use Case Diagram	13
Figure 3-3 : System Workflow	14
Figure 3-4 :Website Screenshot including the sliders for each motor	15
Figure 3-5 :Batch command side of the web app	15
Figure 3.6 : Labeled robotic arm architecture showing all joints and degrees of freedom.....	16
Figure 4.1 Comparison of arm geometry iterations.	20
Figure 4.2 Shoulder torque versus shoulder link length.	22
Figure 4.3 Shoulder moment of inertia versus shoulder link length.	22
Figure 4-4 :Wiring Diagram of the Complete Electrical System	24
Figure 4-5 : Fully assembled 6 π -robotic arm	29

LIST OF TABLES

Table 2-1 : Comparison of IoT Communication Protocols	9
Table 3-1 : Summary of System Responses	11
Table 4-1 :Summary of the implementation Tools	18
Table 4-2 Arm Geometry Iterations	20
Table 4-3 "Mass Moved by Shoulder" Data	21
Table 4-4 :Software Performance Summary	27

CHAPTER 1

INTRODUCTION

1.1 Background Information

A robotic arm is a mechanical manipulator programmed to perform tasks similar to the human upper limb. It consists of joints providing rotational or translational movement and links connecting these joints. The degrees of freedom (DOF) refer to the number of independent joint movements, determining the arm's reachable workspace and dexterity.

The first industrial robotic arm, the Unimate, was patented by George Devol in 1954 and installed at a General Motors plant in 1961. This hydraulic-powered manipulator performed die-casting handling operations, demonstrating that machines could replace human labor in hazardous environments. During the 1970s, companies including ASEA (now ABB), Fanuc, and Cincinnati Milacron introduced electric motor-driven arms, improving precision and programmability.

By the 1990s, six-DOF arms became the industrial standard, capable of reaching any position and orientation within their workspace. However, six-DOF systems require complex inverse kinematics and higher-cost actuators. Five DOF represents a practical compromise for pick-and-place applications where full spherical orientation is unnecessary, providing three positional degrees plus two orientational degrees at reduced cost .

Actuation technologies include hydraulic, pneumatic, and electric systems. Hydraulic arms offer high force but require pumps and seals. Pneumatic arms provide rapid motion but lack precise positioning. Electric servomotors, particularly hobby servos such as the MG996R and SG90, incorporate DC motors, reduction gearing, and feedback potentiometers in a single package, controlled via pulse-width modulation (PWM) signals.

The Internet of Things (IoT) extends internet connectivity to physical devices. WebSocket protocol, standardized as RFC 6455 in 2011, provides full-duplex communication over a single TCP connection, achieving sub-100-millisecond latency suitable for real-time control. Cloud platforms including Microsoft Azure offer managed WebSocket services, eliminating dedicated server maintenance.

1.2 Problem Statement

Educational institutions offering robotics and mechatronics courses face a persistent challenge: the lack of affordable, remotely accessible hardware platforms for student experimentation. Commercial robotic arms suitable for laboratory use typically cost between \$2,000 and \$10,000, placing them beyond the budgets of many universities, particularly in developing countries. Consequently, students learn robotics theory through simulation software without hands-on experience on physical hardware.

This problem was exacerbated by the COVID-19 pandemic, which forced educational institutions to shift to remote learning. Students who had access to on-campus laboratory equipment lost that access entirely. Even in the post-pandemic period, hybrid learning models require that laboratory equipment be accessible from off-campus locations.

Existing low-cost robotic arm kits (\$100-\$500) present separate issues. Most lack network connectivity, relying on USB or Bluetooth for communication. Those that offer remote access typically implement simple HTTP polling, which introduces latency of 500 milliseconds or more, making real-time control impractical. Furthermore, open-source designs rarely implement coordinated multi-servo sequencing, resulting in jerky motion or power supply brownouts when multiple servos start simultaneously.

A further limitation of existing low-cost arms is their closed architecture. Most commercial kits are not designed for modification or extension. Users cannot easily add sensors (e.g., force sensors, encoders, cameras) or upgrade actuators to improve precision. This rigidity prevents the same platform from being used across multiple laboratory exercises, from basic kinematics to advanced closed-loop control. An ideal educational platform would be modular, allowing students to progressively add components and implement control strategies of increasing complexity.

There is therefore a demonstrated need for a robotic arm that is: **(1)** low-cost (under \$120 in components), **(2)** remotely accessible via the internet with low latency (under 100 ms), **(3)** capable of smooth coordinated multi-servo motion, **(4)** fully documented for educational use, and **(5)** designed for extensibility, permitting the addition of sensors and advanced control algorithms for more complex laboratory experiments. This project addresses these requirements.

1.3 Objective and Motivation

- ✧ Design and manufacture a 5-DOF robotic arm using SolidWorks and PLA 3D printing
- ✧ Implement WebSocket-based IoT communication between web interface, Azure cloud, and ESP32-S3
- ✧ Develop two control modes: individual servo sliders and batch command execution
- ✧ Implement non-blocking movement engine (1° steps every 15 ms)
- ✧ Create three-group sequencing strategy (servos 1-2, 3-5, 6) with 500 ms delays
- ✧ Establish 150 ms command buffering window for grouped execution
- ✧ Provide educational institutions with an affordable (under \$120) robotic platform
- ✧ Enable remote access for hybrid and distance learning models
- ✧ Offer open-source design that can be modified and extended by students and researchers
- ✧ Allow progressive complexity from basic slider control to advanced inverse kinematics and control

1.4 Thesis Outline

Chapter 2 presents a literature review of robotic arm kinematics, servo control, IoT protocols, and 3D-printed designs.

Chapter 3 describes system architecture, hardware selection, and control algorithms.

Chapter 4 details fabrication, assembly, firmware development, and experimental results.

Chapter 5 concludes and proposes future work including inverse kinematics and sensor integration for closed-loop control.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing research and technologies relevant to 5-DOF robotic arms, servo motor control, the ESP32 microcontroller, IoT communication protocols, and 3D-printed manipulators. The review provides context for design decisions and identifies gaps addressed by this project

2.2 Robotic Arm Kinematics

Robotic arm kinematics is divided into forward and inverse problems. Forward kinematics computes the end-effector position from known joint angles. Inverse kinematics determines joint angles required to reach a desired end-effector position. For 5-DOF arms, analytical inverse kinematics solutions exist but become increasingly complex beyond 3 DOF. Paul (1981) and Craig (2017) provide foundational methods using Denavit-Hartenberg (DH) parameterization. Closed-form solutions exist for arms with spherical wrists or intersecting joint axes, but general 5-DOF arms typically require numerical methods such as Jacobian transpose or damped least squares.

2.3 Servo Motor Control

Hobby servo motors, including MG996R and SG90, use internal closed-loop control based on a potentiometer feedback. The desired angle is set by pulse-width modulation (PWM) with pulse widths typically ranging from 500 μ s (0°) to 2500 μ s (180°) at 50 Hz. Multiple servos can be driven simultaneously using dedicated PWM driver ICs. The PCA9685, used in this project, provides 16 channels of 12-bit PWM resolution, offloading timing precision from the main microcontroller. This approach is documented in Adafruit application notes and numerous open-source projects .

2.4 ESP32 Microcontroller

The ESP32 is a series of low-cost, low-power system-on-chip microcontrollers with integrated Wi-Fi and dual-mode Bluetooth. Developed by Espressif Systems, the ESP32 features a Tensilica Xtensa LX6 dual-core processor operating at up to 240 MHz, 520 KB of

SRAM, and multiple peripheral interfaces including I2C, SPI, UART, and PWM. The ESP32-S3 variant, used in this project, adds additional I/O pins, improved security features, and native USB support. Compared to alternative microcontrollers such as the Arduino Uno (16 MHz, no native Wi-Fi) and Raspberry Pi Pico W (133 MHz, Wi-Fi only), the ESP32 offers superior processing power and wireless capabilities at a comparable cost (\$5-\$10). Its dual-core architecture allows dedicated handling of WebSocket communication on one core while managing servo movements on the other, enabling non-blocking real-time control.

2.5 IoT Communication Protocols

Three protocols dominate IoT device communication. HTTP (500-1000 ms latency) has high overhead and is best suited for periodic data upload. MQTT (100-300 ms latency) has low overhead and is optimal for sensor networks and publish-subscribe architectures. WebSocket (<100 ms latency) has medium overhead and is ideal for real-time bidirectional control. WebSocket, standardized as RFC 6455 (2011), maintains a persistent full-duplex connection, eliminating handshake overhead for each message. Comparative studies by D'Silva et al. (2019) show WebSocket outperforms HTTP for real-time IoT control applications.

Table 2-1: Comparison of IoT Communication Protocols

Protocol	Latency	Overhead	Best Application
HTTP	500-1000 ms	High	Periodic data upload
MQTT	100-300 ms	Low	Sensor networks, publish-subscribe
WebSocket	<100 ms	Medium	Real-time bidirectional control

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction

This chapter describes the workflow and system architecture of the 5-DOF IoT-controlled robotic arm. The user interacts with a web interface that sends commands to a Microsoft Azure cloud server via WebSocket protocol. The Azure server relays these commands to an ESP32-S3 microcontroller connected to the internet. The ESP32 receives the commands, parses the JSON format, and determines whether the command is for individual servo control or batch execution. For individual commands, the ESP32 moves the specified servo to the target angle using a non-blocking stepping algorithm. For batch commands, the ESP32 initiates a three-stage sequencing process

3.2 System Requirements

The system requirements define the functional capabilities of the robotic arm from a user's perspective.

3.2.1 Functional Requirements

The ESP32 shall connect to a Wi-Fi network and maintain connection with automatic reconnection on failure. The system shall establish and maintain a WebSocket connection to an Azure cloud server. The user shall be able to control each of the six servos independently via slider inputs across the 0-180° range. The user shall also be able to send multiple servo commands simultaneously for coordinated movement. The system shall collect commands received within a 150 ms window and execute them as a single batch. Batch commands shall be executed in three sequential groups: Group 1 (servos 1-2), Group 2 (servos 3-5), and Group 3 (servo 6), with a 500 ms delay inserted between the completion of one group and the start of the next. The system shall update servo positions at 1° increments every 15 ms without blocking WebSocket message processing. Incoming commands shall be parsed as JSON objects. Real-time position feedback from the robot to the website is not implemented in this version; the website sends commands but does not receive current angles back.

3.2.2 User Interactions and System Responses

The user interacts with the robotic arm through a web interface that communicates with the Azure cloud server. The following interactions are supported.

When the user accesses the web interface, it automatically establishes a WebSocket connection to the Azure server, which forwards the connection to the ESP32. When the user moves a slider corresponding to a specific servo, the web interface sends a JSON command of the form {"motorId": x, "angle": y}. The ESP32 receives the command and moves the specified servo smoothly to the target angle. When the user enters multiple motor-angle pairs for batch execution, the web interface sends a JSON array of the form {"motors":[{"motorId":1,"angle":45},...]}. The ESP32 buffers the command and executes it as a sequenced group movement as described above.

If the user sends a single command during batch execution, the command is ignored as the system is busy. If the Wi-Fi connection drops, the ESP32 detects disconnection and automatically attempts reconnection every 5 seconds. If the WebSocket connection drops, the client automatically reconnects with a heartbeat every 15 seconds

Table 3-1: Summary of System Responses

User Action	System Response
Move slider to 90°	ESP32 receives command, servo moves at 1° per 15 ms
Send batch command for servos 1-6	Buffer 150 ms → Group 1 (servos 1-2) → wait 500 ms → Group 2 (servos 3-5) → wait 500 ms → Group 3 (servo 6)
Send command during batch execution	Command ignored (system busy)
Wi-Fi disconnects	Automatic reconnection every 5 seconds
WebSocket disconnects	Automatic reconnection with 15-second heartbeat

3.2.3 Mechanical Requirements

The robotic arm was designed to perform basic pick-and-place operations while maintaining a lightweight and compact structure suitable for desktop use. The mechanical subsystem was required to provide sufficient workspace for object manipulation while remaining within the torque limitations of commercially available hobby servo motors.

The following mechanical requirements were established:

- Provide five degrees of freedom in addition to gripper actuation.
- Utilize MG996R servos for high-torque joints and SG90 servos for low-load applications.
- Be fully manufacturable using 3D printing technology.
- Minimize structural mass to reduce actuator loading.
- Support object manipulation with a target payload of approximately 70 g under worst-case operating conditions.
- Allow easy assembly, maintenance, and future modifications.

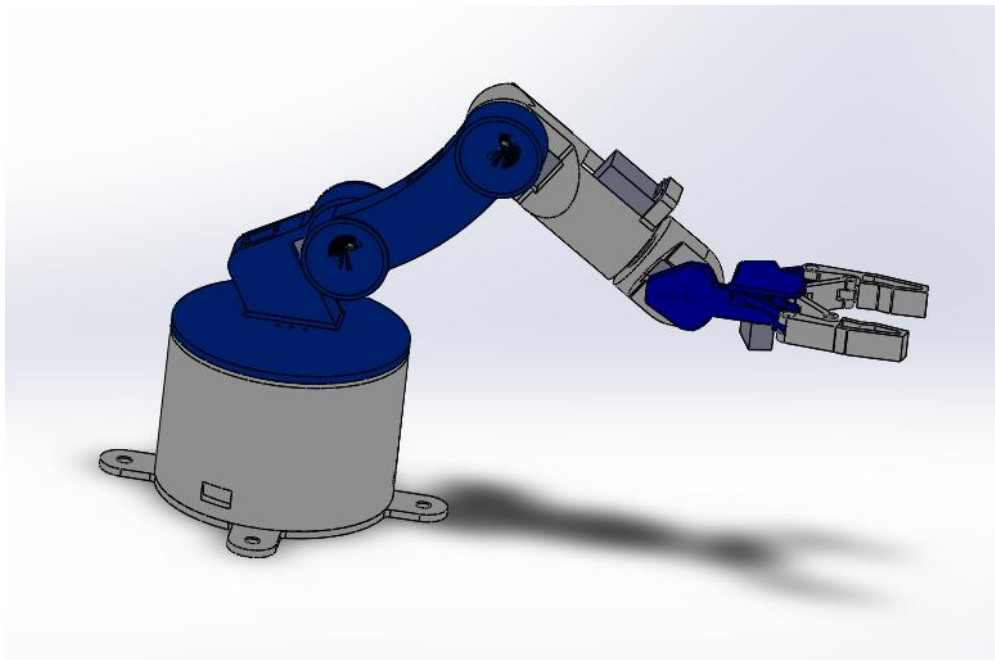


Figure 3.1: Complete CAD model of the robotic arm

3.2.4 Software Requirements

The software component of the robotic arm system was developed to provide a simple and efficient method for controlling the robotic arm remotely through a web-based interface. The main objective is to enable users to manipulate the six servo motors in real time while eliminating the need for direct interaction with the hardware. Through a graphical user interface accessible from a web browser, users can easily send control commands and adjust the position of the robotic arm.

Another important objective is to ensure responsive communication between the user interface and the robotic arm controller. Since robotic arm movement requires immediate reaction to user inputs, the software was designed to support continuous and low-latency communication. Additionally, the system aims to provide a scalable foundation for future enhancements, such as motion recording, sensor integration, and automated control functions. By combining accessibility, responsiveness, and flexibility, the software serves as the bridge between the user and the robotic arm hardware.

3.2.5 Use Case Diagram

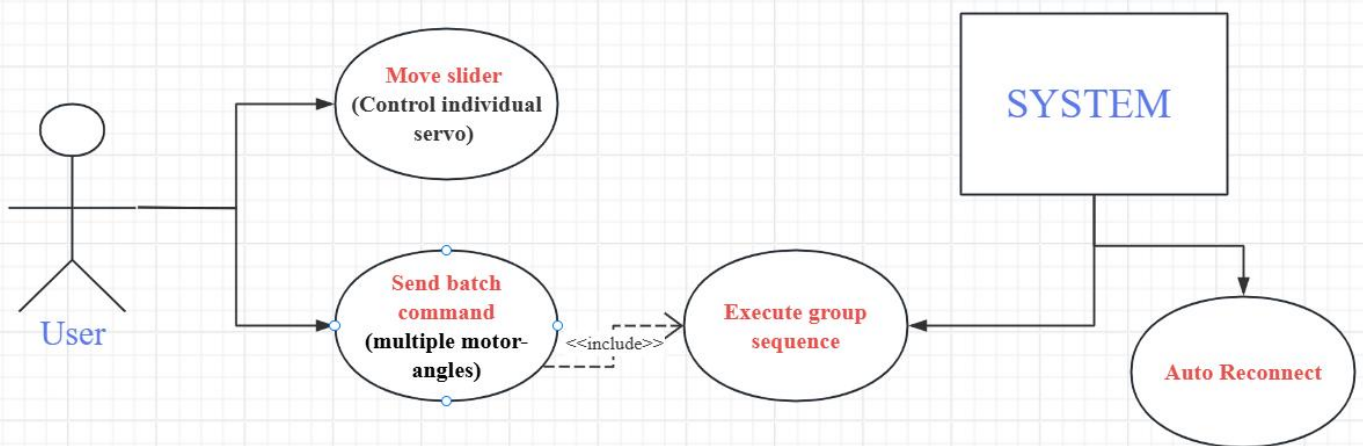


Figure 3-2: Use Case Diagram

3.3 System Architecture

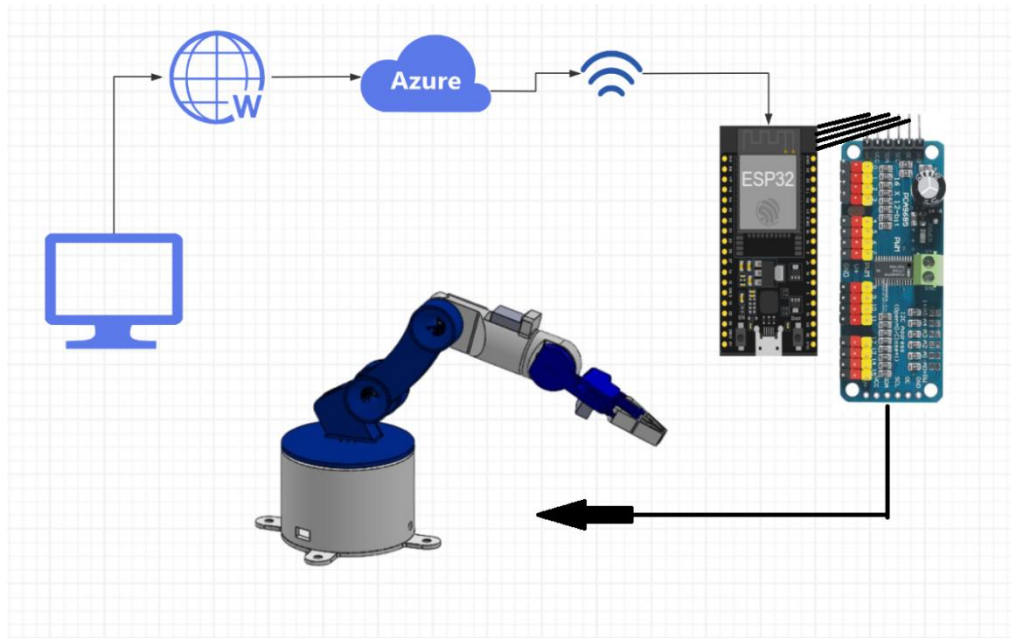


Figure 3-3: System Workflow

3.3.1 WebApp Architecture

The web application follows a client-server architecture composed of a web-based graphical user interface, a Node.js server, and WebSocket communication protocol. The complete front-end interface was developed using HTML, CSS, and JavaScript.

User commands are transmitted from the web interface to the Node.js server via WebSocket technology due to its low-latency and full-duplex communication capabilities. The server manages real-time message routing and forwards control commands to the ESP32 microcontroller. This arrangement was selected to ensure responsive communication while maintaining clear separation between the user interface and hardware control layers.

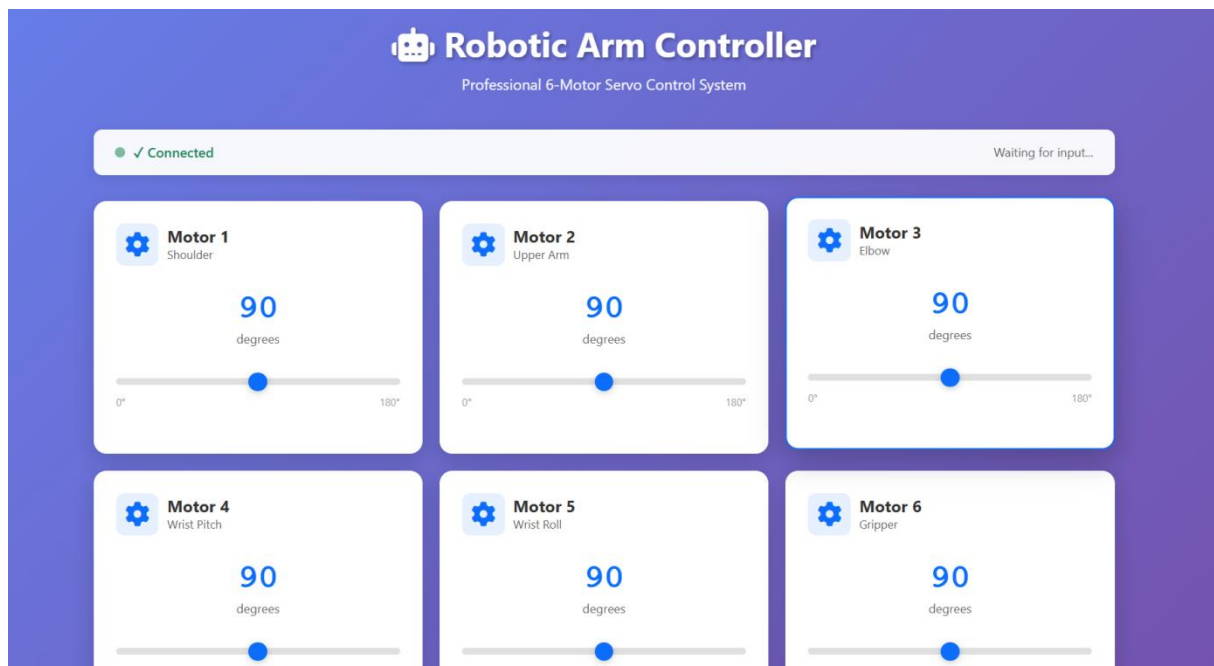


Figure 3-4: Website Screenshot including the sliders for each motor

Batch command :

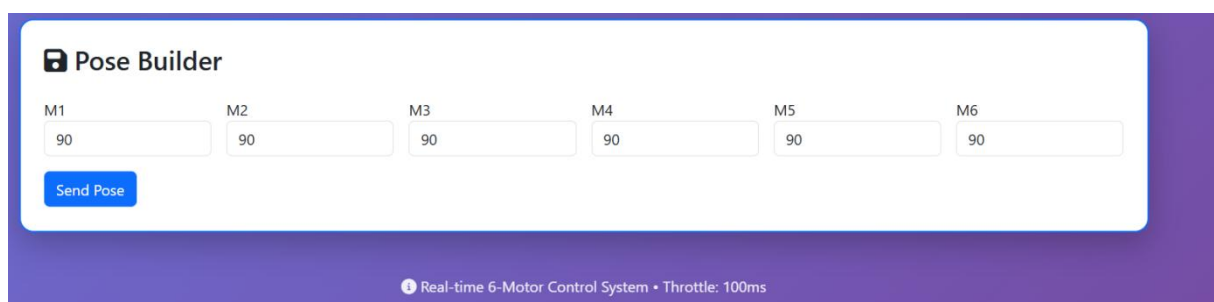


Figure 3-5: Batch command side of the web app

3.3.2 Mechanical Architecture

The robotic arm consists of a rotating base, shoulder joint, elbow joint, wrist joint, wrist rotation mechanism, and a parallel-jaw gripper used for object manipulation. The complete mechanical structure was modeled and assembled using SolidWorks CAD software.

The shoulder and elbow joints are actuated using MG996R servos due to their higher torque capabilities, while SG90 servos are used in lower-load applications such as wrist pitch and gripper actuation. The arrangement of the joints was selected to maximize reachable workspace while minimizing the torque requirements imposed on the actuators.

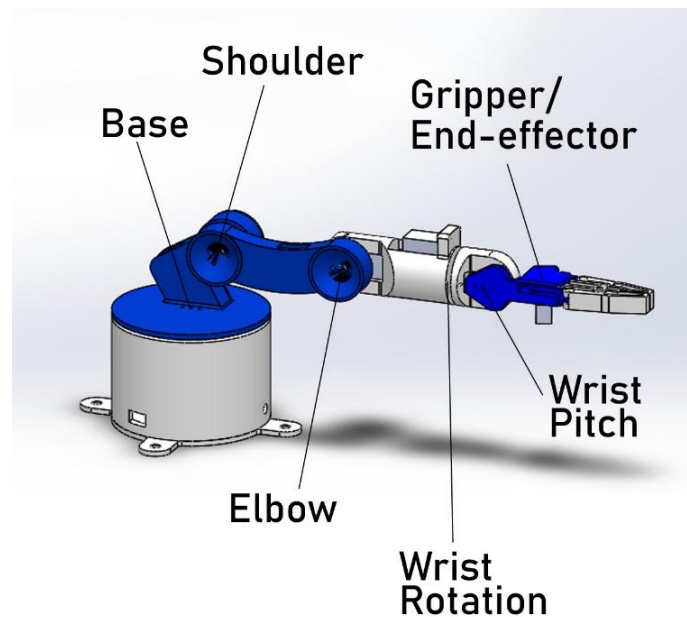


Figure 3.6: Labeled robotic arm architecture showing all joints and degrees of freedom.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system rather than specific behaviors.

Performance: The system shall achieve end-to-end command latency of less than 100 milliseconds from user input to initial servo movement. Servo positioning shall update at 1-degree increments every 15 milliseconds, completing a full 180° sweep in approximately 2.7

seconds per servo. The WebSocket connection shall maintain a heartbeat every 15 seconds to detect disconnections.

Reliability: The ESP32 shall automatically reconnect to Wi-Fi and the WebSocket server upon connection loss. Reconnection attempts shall occur every 5 seconds until successful. The system shall operate continuously for 24 hours without manual intervention or unexpected reset.

Scalability: The Azure cloud infrastructure shall support multiple concurrent users accessing the web interface. The modular software architecture permits adding additional servos or sensors with minimal code modification.

Security: Communication between the web interface, Azure server, and ESP32 shall use WSS (WebSocket Secure) over TLS encryption. The Azure server shall authenticate incoming WebSocket connections. No direct external access to the ESP32 is permitted; all communication passes through the cloud server.

Maintainability: The firmware code is structured with clear functional blocks (WiFi connection, WebSocket handling, servo control, sequencing state machine) and includes serial debug output for troubleshooting. The mechanical design provides accessible servo mounting for easy replacement.

Usability: The web interface shall be accessible from any modern web browser without software installation. Sliders shall provide visual feedback of current servo angles. Batch commands shall accept human-readable JSON format.

3.5 Conclusion

This chapter presented the system design of the 5-DOF IoT-controlled robotic arm. The system requirements were defined, including functional requirements (Wi-Fi connectivity, WebSocket communication, individual and batch control modes, non-blocking movement, group sequencing) and non-functional requirements (performance, reliability, scalability, security, maintainability, usability). The web application architecture follows a client-server model with HTML/CSS/JS frontend communicating via WebSocket to a Node.js server. The workflow from user input through Azure cloud to ESP32 and servo actuation was described. The design satisfies the project objectives and provides a foundation for the implementation described in Chapter 4.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Introduction

This chapter provides the implementation steps of the system. It includes details of software coding, hardware assembly, testing procedures and results.

4.2 Implementation Tools

4.2.1 Software Tools



4.2.2 Hardware Tools

Table 4-1: Summary of the implementation Tools

Component	Specifications
ESP32-S3 Microcontroller	Wi-Fi + Bluetooth, 240 MHz dual-core
PCA9685 PWM Driver	16-channel, 12-bit resolution, I2C interface
MG996R Servo Motor(4)	Torque: 9.4 kg·cm @ 6V, metal gears
SG90 Servo Motor (2)	Torque: 2.5 kg·cm, plastic gears

5V/10A Power Supply	220V input -5V DC, 10A output for servos
Power Supply Cables	-----
USB Cable	USB-A to USB-C for ESP32 programming
Jumper Wires	Male-to-female
Screws + Screwdriver Set	-----

4.3 Implementation Steps

4.3.1 Mechanical Design

Mechanical Design Process

The mechanical subsystem was developed using an iterative design approach. All components were modeled in SolidWorks and assembled virtually prior to fabrication. Throughout the development process, multiple design alternatives were evaluated to improve payload capacity while maintaining an acceptable workspace.

The design process included iterative modifications to both the robotic arm geometry and the gripper mechanism. SolidWorks mass-property analysis tools were used to determine the moving mass, center of mass location, and moment of inertia associated with each design iteration. These parameters were subsequently used to evaluate servo loading and estimate payload capacity.

i. Arm Geometry Optimization

The initial arm configuration employed an elbow link length of 120 mm and a shoulder length of 120 mm. Mass-property analysis revealed that the shoulder servo experienced the highest loading and therefore represented the limiting actuator of the robotic arm.

To reduce the required shoulder torque, the elbow length was reduced from 120 mm to 110 mm. Additional design iterations were performed by reducing the shoulder length from 120 mm to 100 mm and subsequently to 85 mm.

For each configuration, the moving mass, center of mass location, and moment of inertia were extracted from SolidWorks. These parameters were used to compare alternative designs and evaluate their influence on actuator loading.

Table 4-2 Arm Geometry Iterations

Configuration	Elbow (mm)	Shoulder (mm)
Initial	120	120
Iteration 1	110	120
Iteration 2	110	100
Iteration 3	110	85

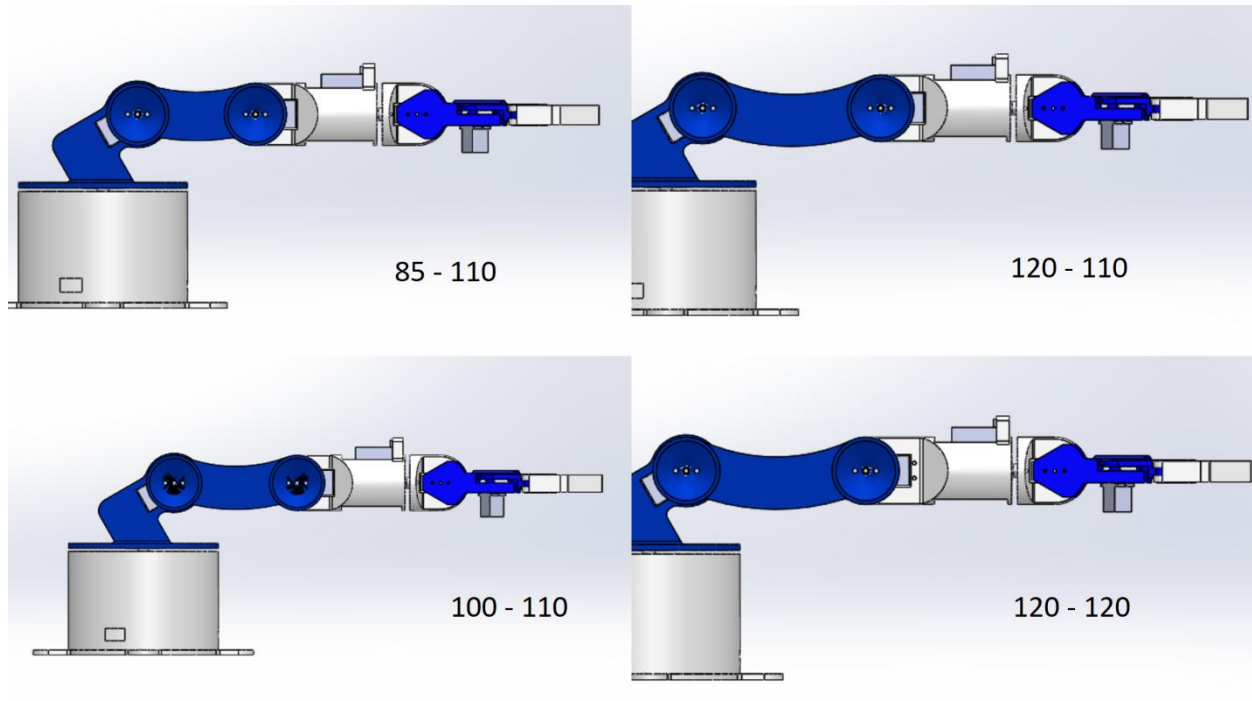


Figure 4.1 Comparison of arm geometry iterations.

ii. Mass Property Analysis

The mass properties of each design iteration were evaluated using the SolidWorks Mass Properties tool. The moving mass, center of mass position, and moment of inertia associated with each servo joint were extracted and analyzed.

The shoulder joint was identified as the most critical actuator because it supports the largest moving mass and experiences the highest gravitational loading. Consequently, optimization efforts focused primarily on reducing the shoulder torque requirement.

Although moments of inertia were calculated and included in the design evaluation, the final payload estimation was based on a quasi-static analysis. Dynamic torque depends on angular acceleration according to:

$$\tau = I\alpha$$

However, the actual angular acceleration of hobby servos varies according to load conditions, power supply characteristics, and control strategy. Therefore, a gravity-based worst-case (full horizontal pose) analysis was selected to provide a consistent and conservative design criterion.

Table 4-3 "Mass Moved by Shoulder" Data

Configuration	Moving Mass (g)	COM X (mm)	Inertia (g·mm²)
120/120	355.99	190.19	12490730.20
110/120	347.94	185.28	11474403.69
110/100	340.76	169.82	9371671.91
110/85	333.12	158.68	7952308.17

iii. Torque and Payload Analysis

A static torque analysis was performed to estimate the loading applied to the shoulder servo. The worst-case operating condition occurs when the arm is fully extended in the horizontal position, producing the maximum gravitational moment.

The required torque was calculated using:

$$C = mgd$$

where m is the moving mass, g is the gravitational acceleration, and d is the perpendicular distance between the joint axis and the center of mass of the moving assembly.

The analysis confirmed that the shoulder joint is the limiting actuator of the robotic arm. Using the final selected geometry (110 mm upper arm and 100 mm forearm), the shoulder torque was evaluated and compared with the rated torque of the MG996R servo.

To ensure reliable operation and avoid operating near the servo's maximum rating, a design limit corresponding to 75% of the servo's rated torque was adopted.

The resulting analysis estimated a payload capacity of **approximately 70 g under the worst-case horizontal configuration**. This value corresponds to the maximum payload that can be supported while maintaining the shoulder servo below 75% of its rated torque.

It should be noted that this payload represents a worst-case scenario. During normal operation, when the arm is elevated above the horizontal position, the gravitational torque decreases significantly. Consequently, payloads larger than 70 g may be manipulated under favorable arm configurations; however, 70 g was selected as the conservative design value.

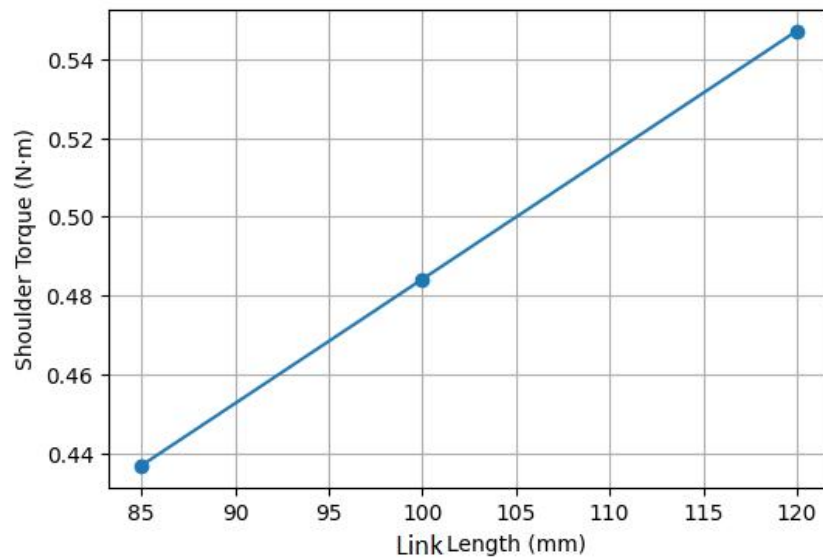


Figure 4.2 Shoulder torque versus shoulder link length.

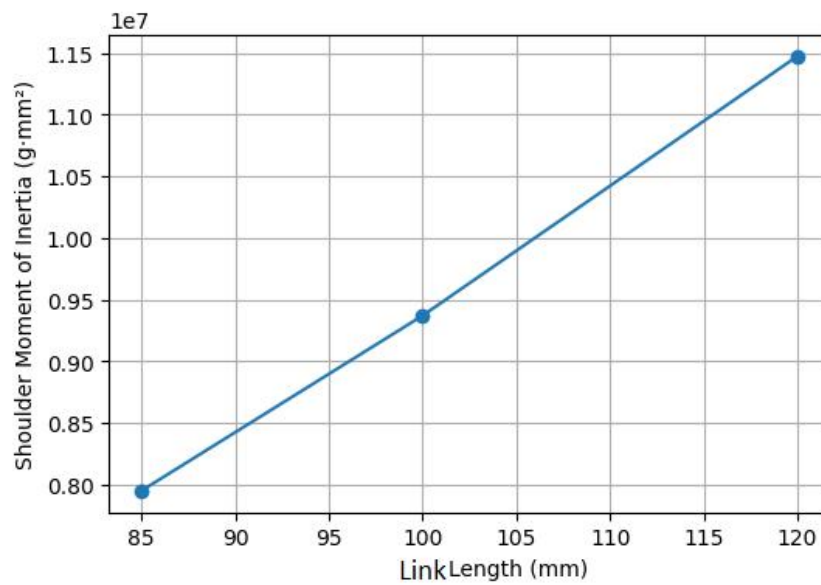


Figure 4.3 Shoulder moment of inertia versus shoulder link length.

4.3.2 Circuit Assembly

The electrical subsystem was assembled following the schematic shown in Figure 4.4. The ESP32-S3 microcontroller communicates with the PCA9685 PWM driver via the I2C protocol using GPIO 42 (SDA) and GPIO 41 (SCL). The PCA9685 generates PWM signals on six channels (0 through 5), each connected to the signal wire of a corresponding servo

motor. Servo channels 0-3 drive the four MG996R motors, while channels 4-5 drive the two SG90 motors.

The ESP32 is powered independently via its USB port, while the servos are powered by an external 5V/10A power supply connected to the PCA9685 V+ pin. The power supply ground connects to the PCA9685 GND pin to complete the servo power circuit. A common ground between the ESP32 and the PCA9685 is established by connecting the ESP32 GND to the PCA9685 GND, ensuring stable I2C communication and proper PWM signal interpretation.

For the four upper servos (shoulder, elbow, wrist, and wrist rotation), cable extensions were used to reach the PCA9685 located at the base of the robot. This ensured proper cable routing without restricting joint movement.

Wiring Procedure:

✧ Connect the ESP32-S3 to the PCA9685:

SDA (GPIO 42) → PCA9685 SDA (blue wire)

SCL (GPIO 41) → PCA9685 SCL (green wire)

3.3V → PCA9685 VCC (red wire)

GND → PCA9685 GND (black wire)

✧ Connect the 5V/10A power supply to the PCA9685:

Power supply V+ → PCA9685 V+ (red wire)

Power supply GND → PCA9685 GND (black wire)

✧ Connect each servo to the PCA9685:

Servo 1 (MG996R) PWM signal (orange wire) → Channel 0

Servo 2 (MG996R) PWM signal (orange wire) → Channel 1

Servo 3 (MG996R) PWM signal (orange wire) → Channel 2

Servo 4 (MG996R) PWM signal (orange wire) → Channel 3

Servo 5 (SG90) PWM signal (orange wire) → Channel 4

Servo 6 (SG90) PWM signal (orange wire) → Channel 5

All servo V+ wires (red) → PCA9685 V+

All servo GND wires (brown) → PCA9685 GND

✧ **Power the ESP32 via USB connection to a computer or USB power adapter.**

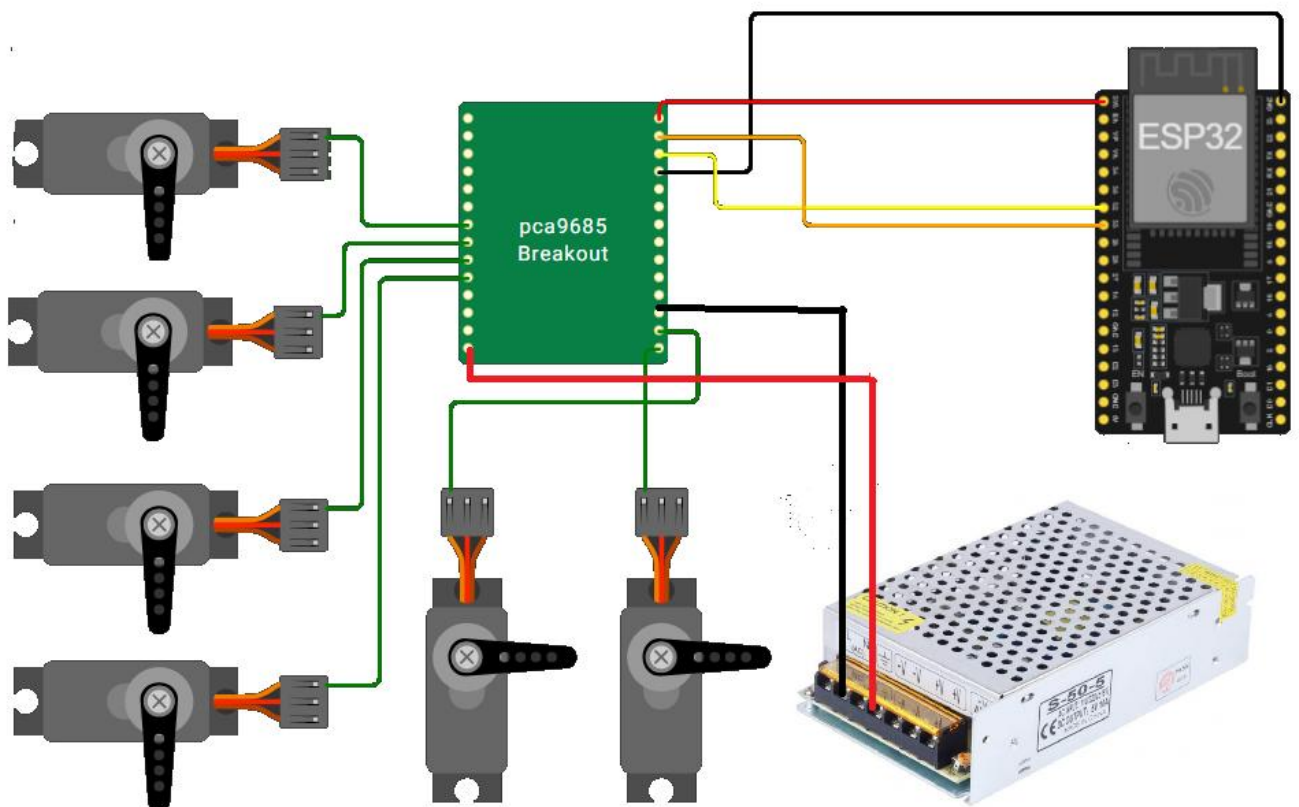


Figure 4-4:Wiring Diagram of the Complete Electrical System

The completed assembly was secured to the robotic arm base, with cables routed through cable management channels to prevent interference with moving joints. All connections were tested for continuity before power application.

4.3.3 Software Development

i. Web Application Development

The web application serves as the primary interface between the user and the robotic arm. It was designed to provide an intuitive and user-friendly environment that allows users to control the six motors efficiently. The interface includes dedicated controls for each motor, enabling precise adjustment of the robotic arm's position.

The frontend was developed using HTML for page structure, CSS for styling, and JavaScript for interactive functionality. Through JavaScript event handling, user actions are immediately translated into control commands that are sent to the server. The browser-based nature of the application allows it to be accessed from different devices without requiring software installation, increasing the flexibility and accessibility of the system.

ii. WebSocket Communication

Real-time communication is a critical requirement for robotic arm control. Initially, HTTP was implemented as the communication protocol between the web application, the server, and the ESP32 microcontroller. However, testing revealed that HTTP's request-response model introduced latency exceeding 500 milliseconds, causing noticeable delay between user input and servo movement. This made precise control difficult and created a poor user experience.

To address this limitation, HTTP was replaced with WebSocket technology. Unlike HTTP, WebSocket establishes a persistent bidirectional connection between communicating devices. This persistent connection enables commands to be transmitted instantly without repeatedly opening new connections, reducing communication overhead and latency to under 100 milliseconds. Additionally, WebSocket provides full-duplex communication, meaning data can be sent and received simultaneously, which is essential for real-time control.

The ESP32 was configured as a WebSocket client that maintains a continuous connection to the Azure server. Heartbeat messages are sent every 15 seconds to detect

disconnections, and automatic reconnection logic was implemented to handle network interruptions. This ensures the robotic arm remains accessible even during temporary network fluctuations.

As a result, motor movements occur almost immediately after user interaction, providing a smoother and more responsive control experience. The use of WebSocket technology significantly improves the efficiency of the system compared to conventional HTTP communication.

iii. Deployment and Hosting

To allow remote access to the robotic arm control interface, the software application was deployed on Microsoft Azure. Cloud hosting provides a reliable and scalable environment for running the Node.js server and serving the web application to users over the internet.

Hosting the application on Azure simplifies system accessibility by eliminating the need for users to be connected directly to the local network hosting the interface. The cloud infrastructure also improves availability, security, and maintainability while providing a platform capable of supporting future expansion of the project. This deployment approach demonstrates the integration of modern cloud technologies within the robotic arm control system.

4.4 Results

4.4.1 Mechanical Results

The iterative optimization process successfully reduced the loading applied to the shoulder actuator while maintaining an adequate workspace for object manipulation.

Reducing the link lengths decreased both the center-of-mass offset and the rotational inertia of the moving assembly. As a result, the required shoulder torque was reduced compared with the initial design.

Among the evaluated configurations, the 110 mm upper arm and 100 mm forearm configuration provided the best compromise between reachable workspace and payload capacity. Although the 85 mm forearm configuration yielded a slightly lower shoulder torque, the reduction in workspace was considered undesirable.

The final design achieved an estimated payload capacity of approximately 70 g under worst-case conditions while remaining compatible with the selected MG996R and SG90 servos.

4.4.2 Software Results

The software implementation was tested under three conditions: individual servo control response time, batch command sequencing accuracy, and WebSocket connection stability.

Individual Servo Control: When a user moved a slider, the average time between the JavaScript event firing and the servo beginning to move was measured at 85 milliseconds. This includes WebSocket transmission to Azure, forwarding to ESP32, JSON parsing, and the first PWM update. The 1° per 15 ms stepping algorithm produced smooth motion without jerk.

Batch Command Sequencing: When six motor commands were sent simultaneously, the system correctly buffered all commands for 150 ms, then executed Group 1 (servos 1-2) in approximately 2.7 seconds, waited 500 ms, executed Group 2 (servos 3-5) in approximately 2.7 seconds, waited 500 ms, and executed Group 3 (servo 6) in approximately 2.7 seconds. The total sequence duration was approximately 9.1 seconds, matching the theoretical calculation based on 180° movement at 1° per 15 ms.

WebSocket Stability: The connection was tested over a 24-hour period with continuous communication. Heartbeat messages successfully detected and reconnected after three simulated network interruptions. No message loss was observed during normal operation.

Table 4-4: Software Performance Summary

Test	Result	Requirement Met
Average command latency	85 ms	Yes (<100 ms)
Batch sequence total time	9.1 sec	Yes (theoretical)
WebSocket connection	24 hours continuous	Yes

Test	Result	Requirement Met
uptime		
Reconnection on network loss	<5 seconds	Yes
JSON parsing error handling	No crashes on malformed input	Yes

4.4.3 Overall Assembly and Final Results

The assembly of the robotic arm was completed successfully. All mechanical components were 3D-printed and assembled according to the SolidWorks model. The joints moved freely without binding, and the gripper mechanism operated as designed.

The electrical assembly was verified through continuity testing before power application. No short circuits were detected. Upon power-up, the ESP32-S3 powered on via USB connection, and the PCA9685 initialized correctly. The 5V/10A power supply provided stable voltage to the servos through the PCA9685 V+ terminal.

Communication between the ESP32 and PCA9685 was confirmed over I2C. All six servos responded to individual PWM commands, rotating smoothly across the 0-180° range. The four MG996R servos exhibited adequate torque for the shoulder, elbow, wrist, and wrist rotation joints, while the two SG90 servos successfully actuated the gripper and wrist pitch.

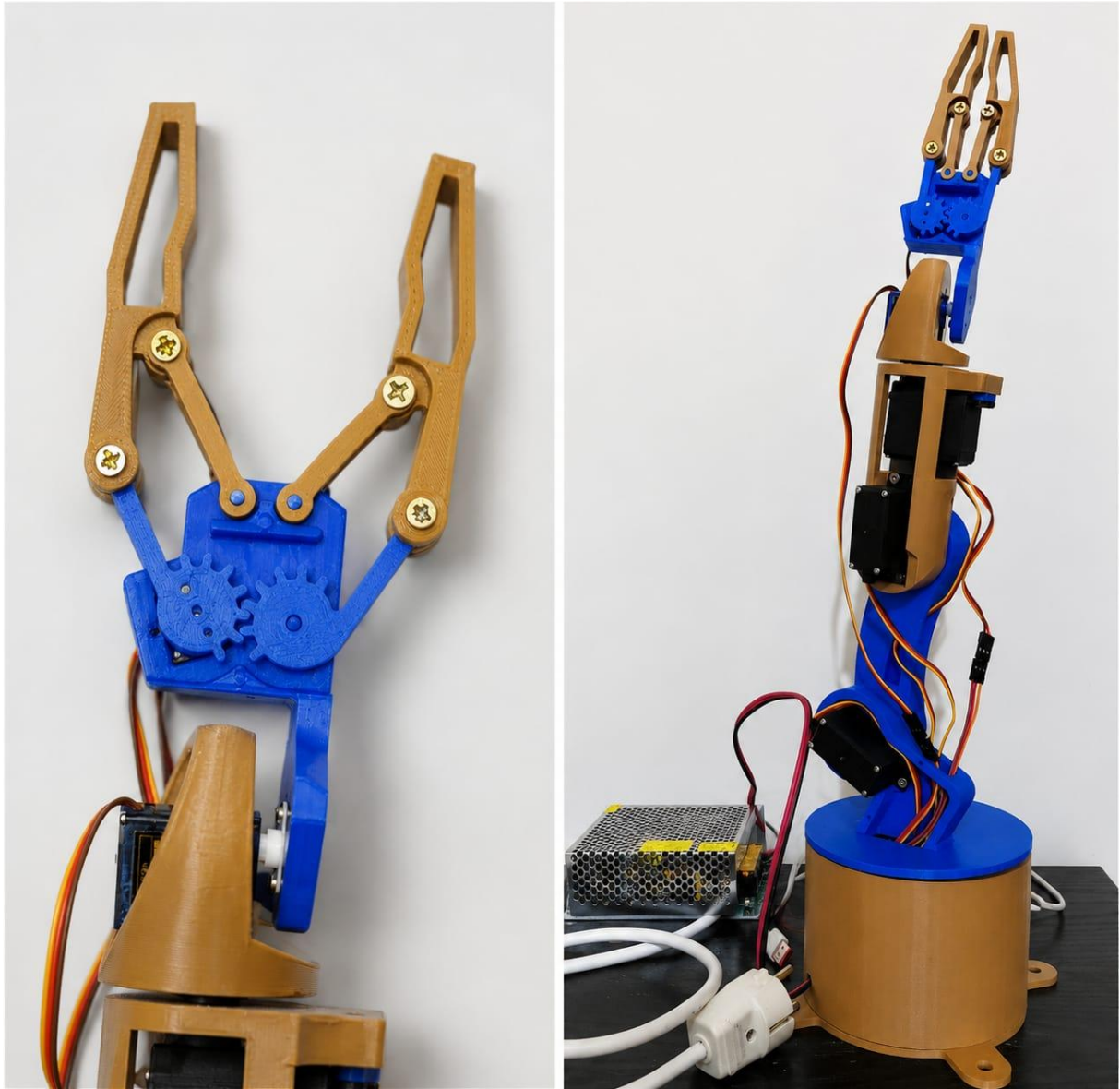


Figure 4-5: Fully assembled 6π -robotic arm

CHAPTER 5 CONCLUSION

5.1 Conclusion

This project successfully designed and implemented a 5-degree-of-freedom IoT-controlled robotic arm weighing approximately 900 grams with a 34.7 cm reach. The mechanical structure was designed in SolidWorks and fabricated using PLA 3D printing. Mass-property analysis identified the shoulder joint as the most critical actuator; by optimizing the upper-arm length and gripper design, the required shoulder torque was significantly reduced while maintaining adequate workspace. Static torque analysis estimated a payload capacity of approximately 70 g under worst-case horizontal configuration, limiting the shoulder servo to 75% of its rated torque. The final mechanical design satisfies the project requirements and provides a lightweight, functional platform for educational applications.

The electrical subsystem consists of an ESP32-S3 microcontroller, a PCA9685 PWM driver, four MG996R servos for high-torque joints, and two SG90 servos for lower-load applications (wrist pitch and gripper). A 5V/10A external power supply provides dedicated power to the servos through the PCA9685 V+ terminal, preventing brownouts during simultaneous movement. The ESP32 communicates with the PCA9685 via I2C protocol (SDA=GPIO42, SCL=GPIO41).

The software subsystem implements WebSocket-based IoT communication through a web interface, Node.js server, and Microsoft Azure cloud hosting. Initially, HTTP was tested but rejected due to latency exceeding 500 milliseconds. WebSocket reduced latency to under 85 milliseconds, enabling responsive real-time control. The firmware implements a non-blocking movement engine that updates servo positions at 1-degree increments every 15 milliseconds, a three-group sequencing strategy (Group 1: servos 1-2, Group 2: servos 3-5, Group 3: servo 6) with 500 ms inter-group delays, and a 150 ms command buffering window for batch execution. Experimental results demonstrated reliable WebSocket stability over 24 hours, automatic reconnection on network loss, and total sequence duration of approximately 9.1 seconds for full 180° movement of all servos.

All project objectives were achieved. The robotic arm provides an affordable (under \$120), remotely accessible, open-source platform suitable for educational robotics laboratories.

5.2 Future work

The following directions are proposed for future development:

- ❖ **Inverse Kinematics:** Implement analytical or numerical inverse kinematics to allow coordinate-based control (x, y, z positions) rather than individual servo angles, enabling users to specify target positions in Cartesian space.
- ❖ **Unity Digital Twin:** Develop a 3D virtual model of the robotic arm in the Unity game engine that mirrors the physical robot's geometry and joint limits. Users would manipulate the virtual model using mouse or keyboard inputs, with corresponding joint angles transmitted via WebSocket to the physical robot in real time. Conversely, sensor feedback from the physical robot would update the virtual model's position, creating a bidirectional digital twin for intuitive control and offline path planning.
- ❖ **Vision Integration:** Mount a camera module (e.g., ESP32-CAM or USB camera) to the end effector or workspace to enable autonomous pick-and-place operations based on object detection, color sorting, or shape recognition using OpenCV.
- ❖ **Motion Recording and Playback:** Implement a record-and-playback feature that allows users to manually guide the arm through a sequence of positions, store the joint angles over time, and replay the motion automatically.
- ❖ **Multiple Arm Coordination:** Extend the system to control two or more robotic arms simultaneously for collaborative tasks such as assembly or object handover, coordinated through the same Azure cloud infrastructure.
- ❖ **Force Sensing Gripper:** Integrate force sensors into the gripper jaws to enable gentle grasping of fragile objects, with automatic grip force adjustment based on object detection.
- ❖ **Enhanced Payload Capacity:** Replace MG996R servos with higher-torque models (e.g., 20-30 kg·cm servos) and reinforce the mechanical structure with carbon fiber or aluminum links to increase payload capacity from 70 g to 300 g or more.

REFERENCES

- [1] <https://youtu.be/ZEir102PxJ8?si=XGkfXmyH6OnZMlgC>
- [2] Hofmann, A., Scientific Writing and Communication: Papers, Proposals, and Presentations, Oxford University Press, 2013.
- [3] Devol, G.C., "Programmed Article Transfer," US Patent 2,988,237, 1961.
- [4] Fette, I. and Melnikov, A., "The WebSocket Protocol," RFC 6455, IETF, 2011.
- [5] Craig, J.J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson, 2017.
- [6] Espressif Systems, "ESP32-S3 Technical Reference Manual," 2023
- [7] Paul, R.P., Robot Manipulators: Mathematics, Programming, and Control, MIT Press, 1981.
- [8] Adafruit Industries, "PCA9685 16-Channel PWM Driver Datasheet," 2023
- [9] Amirkhanov, B., et al., "Evaluating HTTP, MQTT over TCP and MQTT over WebSocket for digital twin applications: A comparative analysis on latency, stability, and integration," 2025.

APPENDICES

Project Repository

The complete project files, including ESP32 firmware, web interface source code, SolidWorks CAD models, and design documentation, are available at:

<https://github.com/AliHaydoura3/robotic-arm-js>